

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 11

The Race and the Semaphore

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. See the race **2**
2.1 Observations 3

3 Part 2. Fix it: mutex, then semaphore **3**
3.1 A mutex around `counter++` 3
3.2 The same lock, made from a semaphore 4
3.3 The price of the lock 4
3.4 Observations 5

4 Part 3. A semaphore as a signal **6**
4.1 Observations 7

5 Part 4. A semaphore as a counter: the parking lot **7**
5.1 Observations 8

6 Optional. No lock at all **9**

7 Conclusion **9**

1 Goal of the work

Two threads that change the same variable at the same time lose work. The goal of this laboratory work is to see this race happen, to fix it with a mutex and with a semaphore, and to use the same semaphore in three different ways: as a lock, as a signal and as a counter.

The work was performed in an Ubuntu 24.04 (arm64) container running on Docker Desktop on macOS, with 8 CPUs visible to the container, kernel 7.0.14-linuxkit and gcc 13.3.0. Every program was compiled with `gcc -Wall -pthread -o NAME NAME.c`, without `-O2`, so that the optimizer does not hide the race.

2 Part 1. See the race

Two threads each add 1 to the same global counter, one million times, with no protection. The correct result is 2 000 000.

Listing 1: `race.c`

```c
#include <stdio.h>
#include <pthread.h>

#define N 1000000

long counter = 0;               /* shared by both threads */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;              /* read, add 1, write back: three steps */
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

Listing 2: Three runs of `race`

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1113300 (expected 2000000)
$ ./race
counter = 1236701 (expected 2000000)
$ ./race
counter = 1013678 (expected 2000000)
```

About half of the additions are lost, and the result is different at every run.

2.1 Observations

Question 1. The three steps of `counter++`.

1. **Load:** read the value of `counter` from memory into a CPU register.
2. **Add:** add 1 to the register.
3. **Store:** write the register back to `counter` in memory.

Question 2. How two threads lose one addition. Suppose `counter` is 5:

Thread A	Thread B	`counter` in memory
load 5 into its register		5
	load 5 into its register	5
add 1, register = 6		5
	add 1, register = 6	5
store 6		6
	store 6	6

Both threads did one addition, but `counter` went from 5 to 6, not 7. B read the old value before A had written the new one, so A's addition was overwritten.

Question 3. Why the result is different at every run. The two threads run on different cores at the same time, and the moment each thread is started, interrupted or moved is decided by the OS scheduler, together with the caches and the other work on the machine. These timings are never exactly the same, so the number of times the two threads collide in the load–add–store window, and therefore the number of lost additions, changes from run to run.

3 Part 2. Fix it: mutex, then semaphore

3.1 A mutex around `counter++`

The line between `pthread_mutex_lock` and `pthread_mutex_unlock` is the critical section: only one thread may be inside it.

Listing 3: The thread function of `mutex.c` (`main` is the same as in `race.c`)

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);      /* only one thread past this line */
        counter++;                      /* the critical section */
        pthread_mutex_unlock(&lock);    /* let the other one in */
    }
    return NULL;
}
```

Listing 4: Three runs of `mutex`

```
$ ./mutex
```

```
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
$ ./mutex
counter = 2000000 (expected 2000000)
```

3.2 The same lock, made from a semaphore

A semaphore is a number that never goes below 0. `sem_wait` takes one unit (and waits if there is none), `sem_post` gives one back. Started at 1, it behaves like a mutex.

Listing 5: The changed parts of `sem.c`

```c
#include <semaphore.h>
sem_t s;

void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);           /* 1 -> 0: I go in; at 0: wait */
        counter++;
        sem_post(&s);           /* 0 -> 1: the next one may go in */
    }
    return NULL;
}

int main(void) {
    sem_init(&s, 0, 1);         /* 0 = shared between threads, 1 = start value */
    ...                         /* create and join the two threads */
    sem_destroy(&s);
    return 0;
}
```

Listing 6: Three runs of `sem`

```
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
$ ./sem
counter = 2000000 (expected 2000000)
```

3.3 The price of the lock

Listing 7: Timing the three versions

```
$ time ./race
counter = 1007224 (expected 2000000)

real    0m0.002s
user    0m0.003s
sys     0m0.000s
$ time ./mutex
counter = 2000000 (expected 2000000)

real    0m0.030s
```

```
user    0m0.030s
sys     0m0.028s
$ time ./sem
counter = 2000000 (expected 2000000)

real    0m0.024s
user    0m0.036s
sys     0m0.009s
```

Because the times are very small, each program was run five times. Table 1 shows the `real` time in seconds.

Program	Run 1	Run 2	Run 3	Run 4	Run 5	Correct?
`race`	0.002	0.002	0.002	0.002	0.002	no
`mutex`	0.028	0.026	0.033	0.032	0.031	yes
`sem`	0.025	0.023	0.023	0.022	0.026	yes
`atomic` (optional)	0.009	0.009	0.009	0.009	0.015	yes

Table 1: Running time of the four versions (seconds)

On this machine the locked versions are about 12–15 times slower than `race`. Unlike on the reference machine of the handout, `sem` came out slightly faster than `mutex` here. The difference is only a few milliseconds and depends on how the C library implements the two locks on this machine. What does not change is that both are much slower than the version without a lock.

3.4 Observations

Question 1. Is the answer always 2 000 000? Yes. All three runs of `mutex` and all three runs of `sem` print `counter = 2000000`. Only one thread at a time can be between lock and unlock (or between `sem_wait` and `sem_post`), so the load–add–store of one thread can no longer be interrupted by the other.

Question 2. What happens with `sem_init(&s, 0, 2)`? I changed the start value to 2 and ran the program three times:

```
$ ./sem2
counter = 1974763 (expected 2000000)
$ ./sem2
counter = 1960864 (expected 2000000)
$ ./sem2
counter = 1975296 (expected 2000000)
```

The result is wrong again. A start value of 2 means that two units are available, so **both** threads can pass `sem_wait` at the same time and be inside the critical section together: the race of Part 1 is back. Fewer additions are lost than in `race`, because each thread spends most of its time in `sem_wait` and `sem_post` and the two threads meet less often exactly at `counter++`, but the result is no longer guaranteed. A semaphore protects a critical section only if it starts at 1.

Question 3. Why is the locked version slower? Every one of the 2 000 000 additions now also performs a lock and an unlock. Each of them is an atomic instruction with a memory barrier, much more expensive than a plain addition. The lock variable is shared,

so its cache line keeps moving between the two cores. When a thread finds the lock taken, it may have to sleep in the kernel (a `futex` system call), which appears as `sys` time: 0.028 s for `mutex`, against 0.000 s for `race`. Finally, the two threads no longer work in parallel: only one of them can be adding at any moment.

4 Part 3. A semaphore as a signal

Here the problem is the *order*: thread B must read the message only after thread A has written it. A semaphore that starts at 0 is a closed door: B waits at `sem_wait` until A calls `sem_post`.

Listing 8: `order.c`

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

char message[32];
sem_t ready;                    /* starts at 0: nothing is ready yet */

void *writer(void *arg) {
    sleep(1);                   /* pretend the work takes a while */
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);           /* 0 -> 1: wake up B */
    return NULL;
}

void *reader(void *arg) {
    printf("B: waiting for the message...\n");
    sem_wait(&ready);           /* blocks while the value is 0 */
    printf("B: I read \"%s\"\n", message);
    return NULL;
}

int main(void) {
    sem_init(&ready, 0, 0);
    pthread_t a, b;
    pthread_create(&b, NULL, reader, NULL);   /* B starts first, on purpose */
    pthread_create(&a, NULL, writer, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    sem_destroy(&ready);
    return 0;
}
```

Listing 9: Output of `order`

```
$ time ./order
B: waiting for the message...
A: message written
B: I read "hello"

real    0m1.005s
```

B prints its first line at once, then nothing happens for one second (A's `sleep(1)`), then A writes the message and B reads it.

4.1 Observations

Question 1. Start value 1: what does B read, and why?

```
$ ./order1
B: waiting for the message...
B: I read ""
A: message written
```

B reads an **empty string**. The semaphore already holds one unit, so B's `sem_wait` takes it and returns immediately, without waiting for A. B reads `message` while A is still sleeping; `message` is a global array, so it starts filled with zeros, which is an empty string. A writes `"hello"` one second later, when B has already finished. The order is no longer guaranteed.

Question 2. What does a semaphore that starts at 0 mean? It means "the event has not happened yet": any thread that calls `sem_wait` blocks until another thread signals with `sem_post` that the event (here, "the message is written") has happened.

5 Part 4. A semaphore as a counter: the parking lot

Six cars share three parking places. The semaphore starts at 3 and counts the free places. The mutex only keeps the variable `inside` and the matching `printf` together.

Listing 10: `parking.c`

```c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define PLACES 3
#define CARS   6

sem_t places;                   /* free places */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;                 /* only for printing */

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);          /* take a place, or wait */
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);

    sleep(1);                   /* parked */

    pthread_mutex_lock(&lock);
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
```

```
    sem_post(&places);          /* give the place back */
    return NULL;
}

int main(void) {
    sem_init(&places, 0, PLACES);
    pthread_t t[CARS];
    for (long i = 0; i < CARS; i++)
        pthread_create(&t[i], NULL, car, (void *)(i + 1));
    for (int i = 0; i < CARS; i++)
        pthread_join(t[i], NULL);
    sem_destroy(&places);
    return 0;
}
```

Listing 11: Output of `parking`

```
$ time ./parking
car 1 enters (inside: 1)
car 2 enters (inside: 2)
car 3 enters (inside: 3)
car 3 leaves (inside: 2)
car 1 leaves (inside: 1)
car 4 enters (inside: 2)
car 5 enters (inside: 3)
car 2 leaves (inside: 2)
car 6 enters (inside: 3)
car 5 leaves (inside: 2)
car 4 leaves (inside: 1)
car 6 leaves (inside: 0)

real    0m2.008s
```

Cars 4, 5 and 6 enter only after another car has left. The run takes 2 seconds: two "waves" of parking, each one second long.

5.1 Observations

Question 1. The largest value of `inside`, and the line that guarantees it. The largest value is **3**. It is guaranteed by `sem_wait(&places);` in `car()`, together with `sem_init(&places, 0, PLACES);` in `main()`, which sets the semaphore to 3. Each car that enters takes one unit; when the value reaches 0, the fourth car blocks in `sem_wait` until a car leaves and calls `sem_post(&places)`. The mutex does not limit the number of cars; it only keeps `inside++`/`inside-` and the matching `printf` together.

Question 2. `PLACES` set to 1: what does the semaphore become?

```
$ time ./parking1
car 1 enters (inside: 1)
car 1 leaves (inside: 0)
car 2 enters (inside: 1)
car 2 leaves (inside: 0)
car 3 enters (inside: 1)
car 3 leaves (inside: 0)
car 4 enters (inside: 1)
car 4 leaves (inside: 0)
```

```
car 5 enters (inside: 1)
car 5 leaves (inside: 0)
car 6 enters (inside: 1)
car 6 leaves (inside: 0)

real    0m6.011s
```

With one place the semaphore can only be 0 or 1, so it becomes a **binary semaphore, which works like a mutex**, the same as in `sem.c` in Part 2. Only one car can be inside at a time, the cars go strictly one after another, and the run takes 6 seconds instead of 2.

6 Optional. No lock at all

For a single counter there is a third way: an atomic operation. `atomic_fetch_add` performs the load, the addition and the store as one indivisible CPU instruction.

Listing 12: The changed parts of `atomic.c`

```c
#include <stdatomic.h>
atomic_long counter = 0;        /* every operation on it is indivisible */

void *add(void *arg) {
    for (int i = 0; i < N; i++)
        atomic_fetch_add(&counter, 1);   /* load, add, store as ONE step */
    return NULL;
}
```

Listing 13: Output of `atomic`

```
$ time ./atomic
counter = 2000000 (expected 2000000)

real    0m0.009s
```

The result is always correct, and the program is about three times faster than the mutex version (0.009 s against about 0.03 s), because there is no lock to take and release and no thread ever sleeps. It is still slower than the wrong `race` version (0.002 s), because the two cores must still pass the cache line of `counter` between them. This works only for a single variable; to protect several operations together a mutex or a semaphore is still needed.

7 Conclusion

Two threads that update the same variable without protection lose work, because the operation `counter++` is three steps and not one, and the scheduler can interleave them in any way. A semaphore is a single tool that solves three different problems depending only on its start value:

Start value	**Use**	**Program**
1	lock (mutual exclusion, like a mutex)	`sem.c`
0	signal (B waits for an event from A)	`order.c`
N	counter of N free resources	`parking.c`

The protection has a price: the locked versions were 12–15 times slower than the unprotected one, and an atomic instruction is a cheaper alternative when only a single variable must be protected.